МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа № 2 по курсу «Операционные системы и архитектура компьютера»

Выполнил: Пантюхин Никита Олегович
Группа: М8О-201БВ-25
Преподаватель: Миронов Евгений Сергеевич
Оценка: _____

Москва, 2026

Содержание

1 Цель работы **2**

2 Постановка задачи **2**

3 Структура проекта **2**

4 Архитектура решения **3**

4.1 Схема межпроцессного взаимодействия 3

5 Ключевые фрагменты реализации **4**

5.1 Обработка обрыва канала 4

5.2 Создание канала и дочернего процесса 4

5.3 Перенаправление stdin и запуск child 5

5.4 Гарантированная запись данных 5

5.5 Проверка и суммирование чисел 6

5.6 Завершение parent и обработка статуса child 7

6 Обработка ошибок и краевых случаев **7**

7 Результаты работы **8**

7.1 Сборка проекта 8

7.2 Базовый сценарий 8

7.3 Некорректный ввод и возможный обрыв IPC 9

7.4 Проверка граничного значения int 9

7.5 Проверка строки большой длины 9

8 Диагностика системных вызовов с помощью strace **10**

9 Вывод **11**

1 Цель работы

Приобретение практических навыков управления процессами в операционной системе и организации обмена данными между процессами посредством каналов (pipe).

В ходе работы необходимо было реализовать многопроцессную программу на языке C для Linux, создать дочерний процесс и организовать межпроцессное взаимодействие средствами POSIX API.

2 Постановка задачи

Для варианта 1 родительский процесс должен создать дочерний процесс. Первой строкой пользователь вводит имя выходного файла. Родительский и дочерний процессы должны быть представлены разными программами.

Далее пользователь вводит строки вида

`число число число<endline>`,

где числа имеют тип `int`, а их количество в строке может быть произвольным. Родительский процесс передаёт введённые строки дочернему процессу через канал `pipe`. Стандартный поток ввода дочернего процесса связывается с каналом. Дочерний процесс вычисляет сумму чисел каждой строки и записывает результат в указанный файл.

Обратный канал из схемы задания для варианта 1 не требуется: по условию он используется только при необходимости, а результат работы дочерний процесс записывает непосредственно в выходной файл.

Программа также должна корректно обрабатывать ошибки системных вызовов и освобождать используемые ресурсы операционной системы.

3 Структура проекта

Структура директории лабораторной работы имеет следующий вид:

```
lab2/
|-- build/
|-- report/
|   |-- report.pdf
|   `-- report.tex
|-- src/
|   |-- child.c
|   |-- io_utils.c
|   |-- io_utils.h
```

```
|   `-- parent.c
|-- CMakeLists.txt
|-- README.md
`-- Лабораторная работа 2.pdf
```

В директории `src/` находится исходный код программы, а в `report/` — исходный файл отчёта в формате LaTeX и собранный PDF. Директория `build/` содержит результаты сборки CMake. Файл `README.md` содержит краткое описание лабораторной работы, инструкции по сборке и запуску. Файл `Лабораторная работа 2.pdf` содержит исходное условие лабораторной работы.

Сборка проекта автоматизирована с помощью CMake. Используется стандарт C17 с GNU/POSIX-расширениями. Для компиляции включены предупреждения `-Wall`, `-Wextra` и `-Wpedantic`.

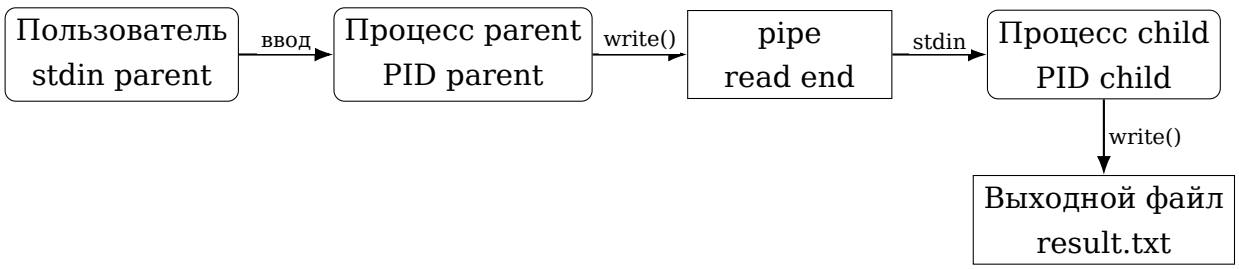
4 Архитектура решения

Программа состоит из двух исполняемых файлов: `parent` и `child`.

- **Parent** считывает имя файла, создаёт канал и дочерний процесс, передаёт пользовательские строки через канал и ожидает завершения дочернего процесса.
- **Child** получает имя файла через `argv`, читает команды из стандартного ввода, связанного с каналом, проверяет входные данные, вычисляет сумму и записывает результат в файл.

Программа `parent` запускается из каталога `build/`, где рядом с ней находится программа `child`. Поэтому для запуска дочерней программы используется простой относительный путь `./child`.

4.1 Схема межпроцессного взаимодействия



Канал создаётся до вызова `fork()`, поэтому его файловые дескрипторы наследуются дочерним процессом. После `fork()` родитель закрывает read-end канала, а дочерний процесс закрывает write-end. Затем дочерний про-

цесс выполняет dup2() для перенаправления своего стандартного ввода на канал и запускает отдельную программу child через execl().

5 Ключевые фрагменты реализации

Ниже приведены ключевые фрагменты исходного кода, отражающие основные механизмы лабораторной работы: обработку обрыва канала, создание процессов и канала, перенаправление стандартного ввода, гарантированную запись данных, проверку входных значений и ожидание дочернего процесса.

5.1 Обработка обрыва канала

```
if (signal(SIGPIPE, SIG_IGN) == SIG_ERR) {
    perror("signal");
    return 1;
}
```

По умолчанию запись в канал без читателя завершает процесс сигналом SIGPIPE. Игнорирование этого сигнала позволяет функции write() вернуть ошибку EPIPE, которую обрабатывает write_all().

5.2 Создание канала и дочернего процесса

```
int pipefd[2];

if (pipe(pipefd) == -1) {
    perror("pipe");
    free(filename);
    return 1;
}

pid_t pid = fork();

if (pid == -1) {
    perror("fork");
    close(pipefd[0]);
    close(pipefd[1]);
    free(filename);
    return 1;
}
```

После успешного fork() одновременно существуют два процесса. В родительском процессе возвращается PID дочернего процесса, а в дочернем

— значение 0.

5.3 Перенаправление stdin и запуск child

```
if (pid == 0) {
    close(pipefd[1]);

    if (dup2(pipefd[0], STDIN_FILENO) == -1) {
        perror("dup2");
        close(pipefd[0]);
        free(filename);
        _exit(1);
    }

    close(pipefd[0]);

    execl("./child", "child", filename, NULL);
    perror("execl");
    free(filename);
    _exit(1);
}
```

После перенаправления исходный read-end канала закрывается, так как стандартный ввод уже ссылается на тот же объект ядра. Затем запускается отдельная программа `child`.

Вызов `dup2(pipefd[0], STDIN_FILENO)` связывает файловый дескриптор 0 дочернего процесса с read-end канала. После этого `getline(..., stdin)` внутри программы `child` получает данные из канала. `execl()` заменяет исполняемую программу в уже существующем дочернем процессе и не изменяет его PID.

5.4 Гарантированная запись данных

Один вызов `write()` не обязан записывать весь переданный буфер, поэтому используется вспомогательная функция `write_all()`.

```
int write_all(int fd, const char *buffer, size_t size) {
    size_t total_written = 0;

    while (total_written < size) {
        ssize_t bytes_written = write(fd, buffer + total_written, size -
   total_written);

        if (bytes_written > 0) {
            total_written += (size_t)bytes_written;
            continue;
        }

        if (bytes_written == -1 && errno == EINTR) {
            continue;
```

```
        }

        if (bytes_written == -1) {
            perror("write");
            return -1;
        }

        fprintf(stderr, "write: wrote 0 bytes\n");
        return -1;
    }

    return 0;
}
```

При EINTR системный вызов повторяется. При частичной записи отправка продолжается с первого ещё не записанного байта. Нулевая запись также рассматривается как ошибка.

5.5 Проверка и суммирование чисел

```
long long sum = 0;
char *token = strtok(line, " \t\r\n");

if (token == NULL) {
    fprintf(stderr, "Empty input line\n");
    return cleanup_on_error(line, file_fd);
}

while (token != NULL) {
    errno = 0;
    char *end = NULL;
    long number = strtol(token, &end, 10);

    if (end == token || *end != '\0' || errno == ERANGE ||
        number < INT_MIN || number > INT_MAX) {
        fprintf(stderr, "Invalid number: %s\n", token);
        return cleanup_on_error(line, file_fd);
    }

    sum += number;
    token = strtok(NULL, " \t\r\n");
}
```

Функция `strtol()` позволяет проверить не только преобразование строки в число, но и наличие лишних символов и выход за диапазон типа `long`. Дополнительно контролируется диапазон типа `int`. Сумма хранится в типе `long long`.

5.6 Завершение parent и обработка статуса child

```
int status;

if (waitpid(pid, &status, 0) == -1) {
    perror("waitpid");
    return 1;
}

if (check_child_status(status) == -1) {
    parent_error = 1;
}
```

`waitpid()` используется для ожидания дочернего процесса, что исключает появление zombie-процесса. Parent проверяет, завершился ли child штатно, с ненулевым кодом или вследствие сигнала.

6 Обработка ошибок и краевых случаев

В программе предусмотрены следующие проверки:

Ситуация	**Реакция программы**
Ошибка `pipe()` или `fork()`	Вывод сообщения через `perror()`, освобождение уже созданных ресурсов.
Ошибка `dup2()` или `execl()`	Вывод причины ошибки; дочерний процесс при ошибках подготовки к `exec` завершается через `_exit(1)`.
Ошибка `open()`	Child выводит причину ошибки и завершается с кодом 1.
Ошибка `getline()`	EOF отделяется от реальной ошибки чтения с помощью `feof()` / `ferror()`.
Пустая строка	Считается некорректной командой.
Некорректный токен	`strtol()` и `end` позволяют обнаружить `abc`, `12abc`, одиночный знак и т.п.
Число вне диапазона `int`	Проверка `INT_MIN` и `INT_MAX`.
Слишком большое число	Проверка `errno == ERANGE`.
Child завершился раньше parent	Если parent попытается выполнить запись после закрытия read-end канала, `write()` возвращает `EPIPE`. `SIGPIPE` игнорируется, поэтому parent может обработать ошибку и затем выполнить `waitpid()`.

Прерывание `write()` сигналом	При `EINTR` системный вызов повторяется.
Закрытие ресурсов	Файловые дескрипторы закрываются, динамические буферы освобождаются.

Получение `EPIPE` не гарантировано при каждой ошибке child и зависит от порядка выполнения процессов. Если parent успел записать очередные строки в буфер pipe до завершения child, он может не получить `Broken pipe`; однако ненулевой статус дочернего процесса всё равно будет обнаружен через `waitpid()`.

7 Результаты работы

7.1 Сборка проекта

Проект был собран GCC 13.3.0 с включёнными предупреждениями компилятора. Сборка завершилась без ошибок и предупреждений.

```
[ 16%] Building C object CMakeFiles/parent.dir/src/parent.c.o
[ 33%] Building C object CMakeFiles/parent.dir/src/io_utils.c.o
[ 50%] Linking C executable parent
[ 50%] Built target parent
[ 66%] Building C object CMakeFiles/child.dir/src/child.c.o
[ 83%] Building C object CMakeFiles/child.dir/src/io_utils.c.o
[100%] Linking C executable child
[100%] Built target child
```

7.2 Базовый сценарий

```
$ ./parent
Enter filename: result.txt
Parent PID: 200568, Child PID: 200713
Enter numbers line by line. Press Ctrl+D to finish.
Child PID: 200713, Parent PID: 200568
1 2 3
10 20 30
-5 10 15
2147483647 -2147483648

$ cat result.txt
6
60
20
-1
```

Вывод PID демонстрирует наличие двух разных процессов. Результаты в файле соответствуют суммам введённых чисел.

7.3 Некорректный ввод и возможный обрыв IPC

```
$ ./parent
Enter filename: result.txt
Parent PID: 201710, Child PID: 201794
Enter numbers line by line. Press Ctrl+D to finish.
Child PID: 201794, Parent PID: 201710
1 2 3
10 abc 20
Invalid number: abc
30 40
write: Broken pipe
Child exited with code 1
```

В данном запуске дочерний процесс обнаружил неверный токен и завершился до следующей записи parent. Поэтому следующая попытка `write()` получила `EPIPE` (`Broken pipe`). При другом планировании процессов parent может успеть записать данные раньше и не получить `EPIPE`; корректность программы при этом сохраняется за счёт последующей проверки статуса child через `waitpid()`.

7.4 Проверка граничного значения int

```
2147483647
2147483648
Invalid number: 2147483648
```

Значение 2147483647 принимается, а следующее значение корректно отвергается как выходящее за диапазон `int`.

7.5 Проверка строки большой длины

Была передана строка из 5000 чисел, равных единице:

```
$ python3 -c 'print("result.txt"); print(" ".join(["1"] * 5000))' | ./parent
...
$ cat result.txt
5000
```

Использование `getline()` позволяет обрабатывать строки динамической длины без фиксированного массива вида `char line[1024]`.

8 Диагностика системных вызовов с помощью strace

Трассировка выполнялась из каталога `build/`, где находятся обе программы:

```
$ strace -f -o trace.txt ./parent
```

Параметр `-f` позволяет отслеживать как родительский, так и созданный им дочерний процесс. Ниже приведён сокращённый фрагмент трассировки:

```
344487 execve("./parent", ["./parent"], ...) = 0
344487 write(1, "Enter filename: ", 16) = 16
344487 pipe2([3, 4], 0) = 0
344487 clone(..., SIGCHLD, ...) = 344560
344487 close(3) = 0
344560 close(4) = 0
344560 dup2(3, 0) = 0
344560 close(3) = 0
344560 execve("./child",
              ["child", "result.txt"], ...) = 0
344560 write(2, "Child PID: 344560, Parent PID: 344487\n", 38) = 38
344487 write(4, "1 2 3\n", 6) = 6
344560 write(3, "6\n", 2) = 2
344487 write(4, "10 20\n", 6) = 6
344560 write(3, "30\n", 3) = 3
344487 close(4) = 0
344487 wait4(344560, <unfinished ...>
344560 close(3) = 0
344487 <... wait4 resumed>[{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) =
   344560
```

В трассировке видно, что `execve()` запускает расположенную рядом программу `./child`. В конце фрагмента `wait4()` сначала блокируется, затем child закрывает выходной файл и завершается; после этого ожидание parent продолжается и возвращает успешный статус. Строка `exit_group()` не попала в сокращённый вывод, поскольку трассировка была отфильтрована по ключевым системным вызовам.

Следует учитывать, что библиотечные POSIX-функции могут быть реализованы через другие системные вызовы ядра Linux. В данном запуске вызову `pipe()` соответствует `pipe2()`, `fork()` — `clone()`, а `waitpid()` — `wait4()`.

Код программы	Наблюдаемый системный вызов
`pipe()`	`pipe2()`
`fork()`	`clone()`
`dup2()`	`dup2()`
`execl()`	`execve()`
`write()`	`write()`
`close()`	`close()`
`waitpid()`	`wait4()`

Трассировка подтверждает создание отдельного процесса, перенаправление стандартного ввода дочернего процесса, обмен данными через pipe, запись результата в файл и корректное ожидание завершения child.

9 Вывод

В ходе лабораторной работы была реализована многопроцессная программа на языке C под Linux. Родительский процесс создаёт дочерний процесс, организует однонаправленный канал передачи данных и перенаправляет стандартный ввод дочернего процесса на read-end канала. Дочерний процесс запускается как отдельная программа, обрабатывает произвольное количество целых чисел в каждой строке, вычисляет их сумму и записывает результаты в файл.

В программе реализована проверка основных ошибок системных вызовов, обработка некорректных входных данных и чисел вне диапазона `int`, обработка обрыва IPC, освобождение файловых дескрипторов и динамической памяти, а также ожидание дочернего процесса через `waitpid()`. Программа `child` запускается по простому относительному пути `./child`, поэтому запуск выполняется из каталога `build/`. Работа основных системных механизмов дополнительно подтверждена трассировкой `strace`.

Репозиторий:

https://github.com/nikitapantyukhin/operating-systems-and-computer-architecture-y2-s3/tree/main/lab2